Code559

Front-End Developer Course

MODULE 4

# Forms and Accessibility

Forms that send the right data and work for every visitor, with or without a mouse.

Free lesson: 559solutions.com/code559/module-04-forms-accessibility/

---

**In this packet**


1. **The lesson**
2. **Project: A rating card and a Contact page**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**


Powered by 559 Solutions

# About this module

## What this module covers

You learn to build forms with real controls, labels and radio groups, and to test them with the keyboard and style their states. It matters to every visitor: a missing label or name can leave a screen reader user guessing, or lose what a visitor typed. The browser already gives you most of the fix if you use the right elements.

## Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

**Recite to understand.** When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

**Write it down to check yourself.** When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

**Keep one notebook for this course.** Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

## The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

## Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

# Contents


**About this module** 2

**Part 1: The lesson** 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What are the parts of a form? 5
  - 2. What does a form send, and why does `name` matter? 6
  - 3. What makes radio buttons a group? 7
  - 4. What do `fieldset` and `legend` add? 8
  - 5. What does the `hidden` attribute do? 9
  - 6. How does a keyboard user move through a form? 9
  - 7. What are `:focus-visible`, `aria-current` and a skip link? 10
  - 8. How do custom properties and state selectors style a control you cannot see? 12
  - 9. How do you read a design brief? 13
- Exercise 14
- Common misconception 16
- Summary 17
- Check your understanding 17
- Before next module 17
- Go further 17
- What to keep 18

**Part 2: Project: A rating card and a Contact page** 19

**Part 3: Finishing work in Git** 30

**Part 4: Troubleshooting: five common problems** 32

**Part 5: Self-assessment** 35

**Part 6: Answers** 37

**About 559 Solutions** 41



© 2026 559 Solutions LLC

PART 1

# The lesson

*Forms that send the right data and work for every visitor, with or without a mouse.*

**Time:** about 2 hours (estimate)

**In this module you will learn:**

- The parts of a form: `form`, `label` with `for`, `input` types, `textarea`, `button` and `required`
- The difference between `id` (for the page) and `name` (for the data)
- What a form sends when the browser submits it, and why only named controls send anything
- Radio buttons, where a shared `name` makes the group, and how `required` on one radio covers the whole group
- `fieldset` and `legend`, which give a group a title
- The `hidden` attribute
- Keyboard navigation: Tab between controls, arrow keys inside a radio group
- `:focus-visible`, `aria-current="page"` and the skip-link idea
- CSS custom properties (`:root`, `var()`), the state selectors `:checked`, `:hover` and `:focus-visible`, and a `.visually-hidden` class that hides a control without breaking it
- How to read a design brief and build the HTML before any CSS

---

## Why this matters

A form is where a visitor talks back to your site. If a label is not joined to its input, a screen reader user does not know what to type. If a control has no `name`, its value never leaves the page. If a custom-styled button cannot be reached with the Tab key, a keyboard user cannot use your form at all. None of those bugs show up when you test with a mouse.

The good news is that HTML already does most of the work. Real form controls come with keyboard behavior, validation and screen reader support built in. Your job is to use the right element, join the pieces correctly, and not break what the browser gives you when you add CSS.

By the end of this module your practice contact page is ready to send data, and you have built the HTML and CSS of a small rating card from a written brief.

---

## Before you start

1. Module 3 is done: you can use the viewport tag, `box-sizing: border-box`, and a mobile-first stylesheet.
2. In your course folder (for example `Documents\code559`), next to `my-site`, create a folder called `practice-04`. This is the throwaway folder for the Exercise. Nothing in it is published.


© 2026 559 Solutions LLC

3. Put the mouse aside for a minute. Open any website and press **Tab** a few times. Watch where the focus goes. That is how many people use the web every day.

> **Checkpoint.** You have an empty `practice-04` folder, and you have moved through a page with the Tab key.

---

## Core concepts

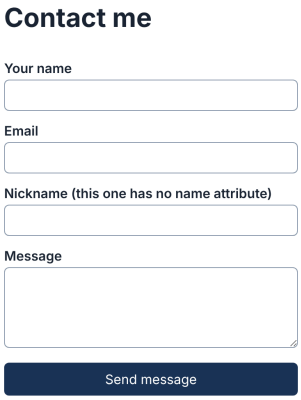
### 1. What are the parts of a form?

A form is a `<form>` element holding controls the visitor fills in and a button that sends them.

Each control is an `<input>` with a `type` (`text`, `email`, `tel`, `radio`, `checkbox` and more), a `<textarea>` for longer text, or a `<button>`. Every control needs a `<label>`. MDN's `<label>` page explains that the label's `for` value is "the `id` of the labelable form control" it belongs to. Two good things follow. Clicking the label focuses or activates the control, which gives a bigger target on a phone. And, in MDN's words, "a screen reader will read out the label when the user is focused on the form input." `required` tells the browser not to send the form until that control has a value. `type="email"` also makes the browser check that the text looks like an email address.

```
<label for="visitor-email">Email</label>
<input type="email" id="visitor-email" name="visitor-email" required>
```


© 2026 559 Solutions LLC




*A contact form at 390 px. Every field has a visible label joined to it with `for` and `id`. The third field is missing something on purpose; Concept 2 shows what. (real: rendered in Chromium at 390 px)*

In step 1 of the Exercise you build this form as `practice-04/contact.html`. When you do, click each label's text. If the cursor does not jump into the field, that label's `for` does not match an `id`.

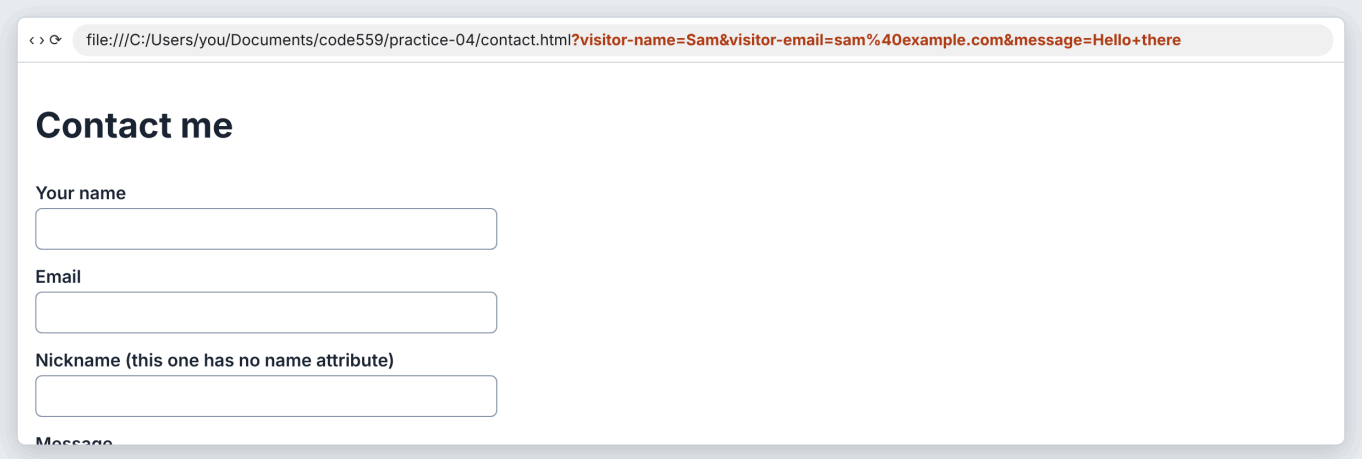
## 2. What does a form send, and why does `name` matter?

`id` and `name` look alike, but they have different jobs. `id` is for the page: labels, CSS and, later, JavaScript. `name` is for the data.

A form with no `method` uses GET. MDN's `<form>` page lists `get` as the default and says the form data is "appended to the `action` URL with a `?` separator." With no `action`, the data goes back to the same page. Each control is sent as `name=value`, and pairs are joined with `&`. A control with no `name` is not sent at all.

The form above was filled in (name, email, a nickname and a message) and submitted for real in Chromium.


© 2026 559 Solutions LLC




*Three pairs arrived. The nickname was typed but is missing, because its input has no `name`. The browser also encoded the text: `@` became `%40` and a space became `+`. (real output, rendered; captured on Linux, with the folder path changed to the course folder. The page was opened from disk and submitted in Chromium; the browser frame is drawn.)*

Predict: if you remove `name` from the message box too, how many pairs will the address show? (Two.)

### 3. What makes radio buttons a group?

A radio button lets the visitor pick exactly one choice from a few.

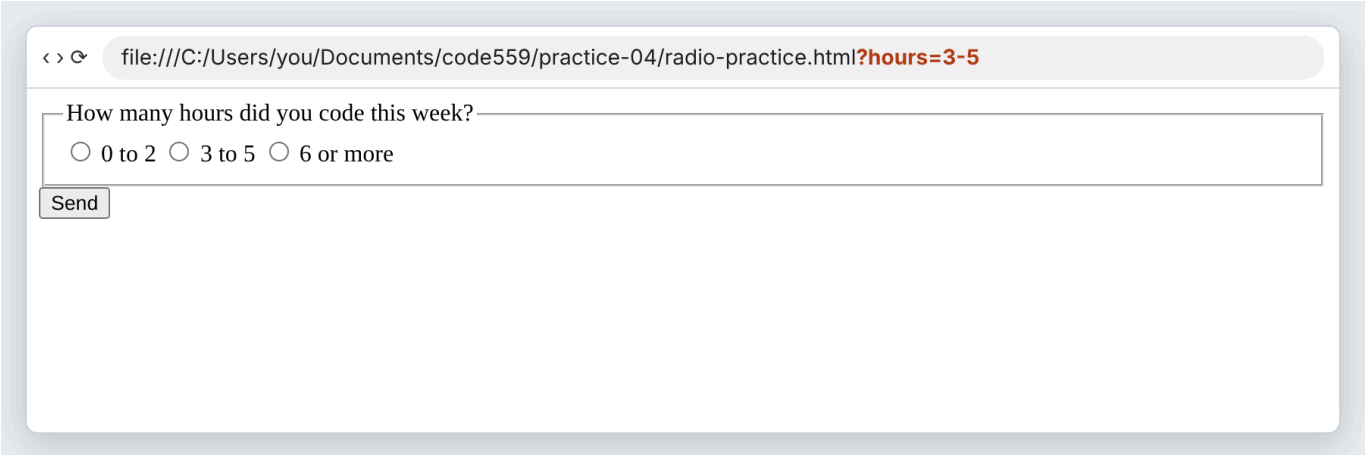
The `name` makes the group. MDN's radio page: "A radio group is defined by giving each of radio buttons in the group the same `name`." Selecting one turns the others off. Each radio still needs its own `id`, so its label can point to it, and its own `value`, because the checked radio sends `name=value`. MDN warns that a radio with no `value` sends the string `on`, which tells you nothing.

`required` is unusual on radios. MDN says that if any radio in a same-named group has `required`, "a radio button in that group must be checked, although it doesn't have to be the one with the attribute applied." So you put `required` on one radio, and it covers the whole group.

```
<input type="radio" name="hours" value="0-2" id="hours-low" required>
<label for="hours-low">0 to 2</label>
<input type="radio" name="hours" value="3-5" id="hours-mid">
<label for="hours-mid">3 to 5</label>
<input type="radio" name="hours" value="6-plus" id="hours-high">
<label for="hours-high">6 or more</label>
```

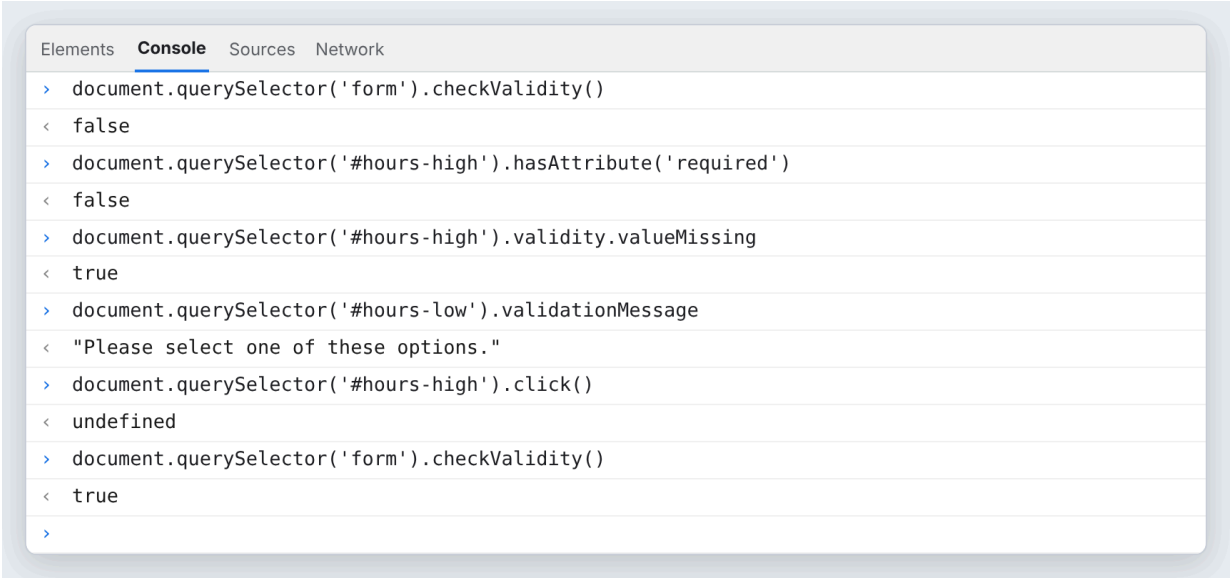
First, a real submit with "3 to 5" chosen:


© 2026 559 Solutions LLC




*One group sends one pair: the group's `name` and the checked radio's `value`. (real output, rendered; captured on Linux, with the folder path changed to the course folder. The page was opened from disk and submitted in Chromium; the browser frame is drawn.)*

Second, `required` checked in the Console with nothing selected:




*Only the first radio has `required`, yet the third radio also reports `valueMissing: true`. Checking any one radio makes the whole form valid. Pressing Send with nothing chosen left the address unchanged and moved focus to the first radio. (real output, rendered; Chromium 141. The message text may be worded differently in your browser or language.)*

Look at one radio in the code above and explain to yourself, out loud, what its `id`, `name` and `value` each do.[1] Then check yourself against the paragraph above.

### 4. What do `fieldset` and `legend` add?

A sighted visitor sees the question above the radios. A screen reader user moving into the group hears a label like "3 to 5," and nothing about what the numbers mean, unless the group has a title.


© 2026 559 Solutions LLC

MDN's `<fieldset>` page says it is "used to group several controls," with "a nested `<legend>` element providing a caption." Wrap the radios in `<fieldset>`, and put the question in `<legend>` as the first thing inside it. It costs two tags.

In the keyboard picture in Concept 6, the box around the radios is the fieldset, and "How many hours did you code this week?" sits in its border because it is the legend.

Any group of radios or checkboxes you write from now on starts with `<fieldset>` and `<legend>`.

### 5. What does the `hidden` attribute do?

Sometimes a page holds a part it is not showing yet, like a thank-you message that appears after a form is sent.

MDN's `hidden` page says "the browser will not render elements that are in the hidden state," and "if something is marked hidden, it is hidden from all presentations, including, for instance, screen readers." It is also out of the Tab order. One warning from the same page: a CSS `display` rule on the element (for example `display: block`) overrides `hidden`, so do not give a hidden element a `display` value in your stylesheet.

On the skip-link page in Concept 7, a `<section hidden>` holds a button. Chromium measured the section at `offsetHeight` 0 with computed `display: none`. Pressing Tab eight times visited the skip link, four nav links and the visible button, and never reached the hidden button.

Predict: you remove `hidden` from that section. Name two things that change. (It appears, and its button joins the Tab order.)

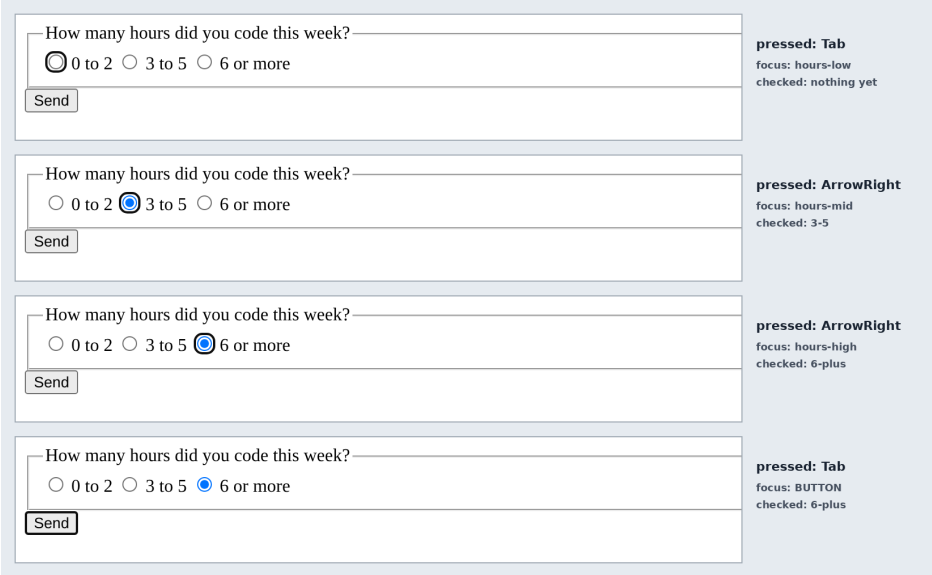
### 6. How does a keyboard user move through a form?

Without a mouse, a visitor uses **Tab** to move forward, **Shift+Tab** to move back, **Space** or **Enter** to press, and the **arrow keys** inside some controls.

A radio group is one stop in the Tab order. Tab enters the group, the arrow keys move the choice inside it, and the next Tab leaves the group. You get all of this for free with real `<input type="radio">` elements. You lose it if you build fake radios out of `<div>`s.

The radio page from Concept 3, driven by keyboard only in Chromium. After each key press the page was captured and asked which element had focus and which radio was checked.


© 2026 559 Solutions LLC




*Tab put focus on the group without choosing anything. Each arrow key moved the focus **and** the choice. The second Tab left the group and went to Send. (real: rendered in Chromium; captured on Linux. Edge and Chrome on Windows use the same engine; check it yourself in step 5 of the Exercise.)*

Open `practice-04/contact.html` (step 1 of the Exercise) in Edge or Chrome, put the mouse aside, and fill in the whole form with the keyboard. Note anything you could not reach.

### 7. What are `:focus-visible`, `aria-current` and a skip link?

Three small tools make keyboard use pleasant: a clear focus ring, a marker for the current page, and a shortcut past the menu.

**`:focus-visible`** is a CSS pseudo-class. MDN says it matches a focused element when the browser decides "the focus should be made evident," so you can style the ring for keyboard users without a ring appearing on every mouse click. In Chromium, a link clicked with the mouse did not match `:focus-visible`; the next link reached with Tab did. Never remove the outline without replacing it.

**`aria-current="page"`** goes on the nav link for the page the visitor is on. It is an ARIA attribute. ARIA (short for Accessible Rich Internet Applications) is a set of roles and attributes that pass extra information to screen readers and other assistive tools when HTML alone has no way to say it. MDN's *ARIA: aria-current attribute* says it marks "the current item within a container or set of related elements," and `page` means the current page. Screen readers announce it, and you can style it with `a[aria-current="page"]`. Square brackets in a selector match an attribute, so this one means "a link whose `aria-current` is exactly `page`."

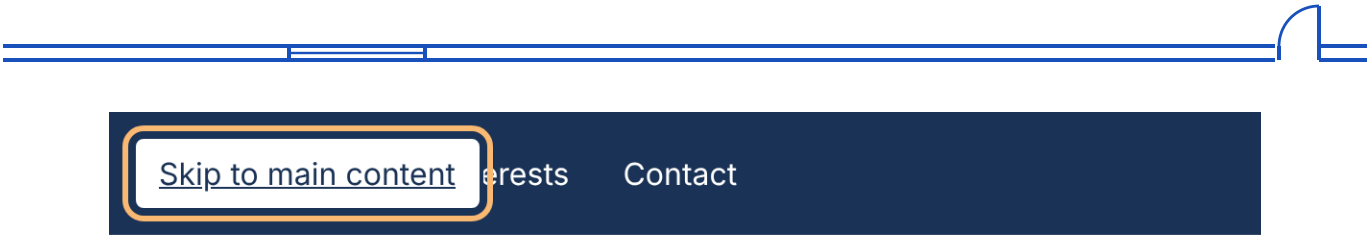
A **skip link** is the first link on the page. It jumps to `<main id="main">`. It sits off screen until it gets focus, so mouse users never see it and keyboard users skip the whole menu with one Enter. In the CSS below, `position: absolute` takes the link out of the normal flow of the page so that `top` can place it:

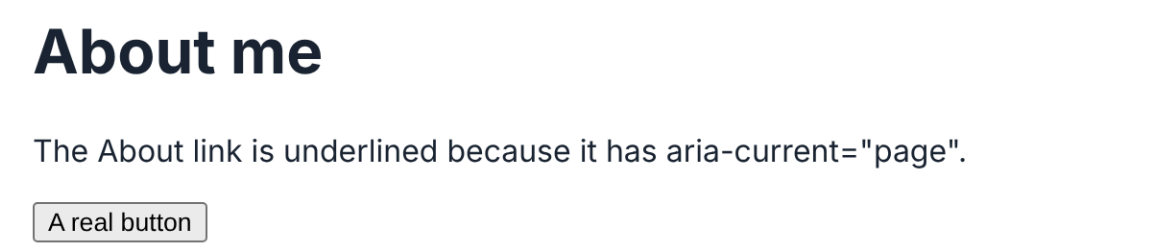

© 2026 559 Solutions LLC

`top: -60px` puts it above the top edge, out of sight, and `top: 12px` brings it back into view when it has focus.

A ring also has to stand out from what is behind it. MDN's *Color contrast* asks for at least 3:1 for parts of the interface. The dark orange ring `#b45309` has 5.0:1 on white but only 2.5:1 on the dark navy bar in the pictures below, so the links on that bar and the skip link, which appears over the bar, get a light orange ring, `#fdba74` (7.3:1 on the navy). Use the light ring only where the background behind the ring is dark: on white, `#fdba74` has only 1.7:1. `outline-color` changes only the ring's color and keeps its width and style.

```
.skip-link { position: absolute; top: -60px; }
.skip-link:focus { top: 12px; }
a:focus-visible { outline: 3px solid #b45309; outline-offset: 2px; }
nav a:focus-visible { outline-color: #fdba74; }
.skip-link:focus-visible { outline-color: #fdba74; }
nav a[aria-current="page"] { text-decoration: underline; font-weight: 700; }
```




*The first Tab on the page reveals the skip link. Pressing Enter jumped to `<main>` (Chromium reported the address ending in `#main`), and the next Tab started from there: it went straight to the button in the main content. (real: rendered in Chromium at 600 px)*


© 2026 559 Solutions LLC




# About me

The About link is underlined because it has aria-current="page".

---

*Three Tabs in: the light orange ring is the `:focus-visible` style for links on the navy bar. About is bold and underlined because it has `aria-current="page"`. (real: rendered in Chromium at 600 px)*

Predict: a site has Home, About and Contact pages, each with the same three nav links. On the Contact page, which link gets `aria-current="page"`? (Only the Contact link. Each page's own link gets it; no other link does.) If you did the packet projects, add `aria-current="page"` to each page's own nav link in `my-site` now.

### 8. How do custom properties and state selectors style a control you cannot see?

A plain radio circle is tiny and hard to style. A common pattern is to hide the circle, style the label as a big button, and let the real radio keep doing its job underneath.

Start with **custom properties**. MDN's *Using CSS custom properties (variables)* says it is "common practice" to define them on `:root` so they can be used everywhere. In HTML, `:root` matches the `<html>` element, which holds the whole page. You read the values with `var()`. Names start with two dashes and are case sensitive.

Next, hide the radio with a **`.visually-hidden`** class. It shrinks the element to one clipped pixel, so it is invisible but still in the page: it still takes focus, is still announced by screen readers, and is still sent with the form. `display: none` and `hidden` work differently: they take the control out of sight, out of the Tab order and away from screen readers. A checked or filled control hidden that way is still sent with the form, but a keyboard or screen reader user can no longer reach it.

Finally, style the label from the radio's **state**. Put each input right before its label. `+` selects the next sibling, so `input:checked + label` means "the label right after a checked radio."

```
:root {
  --ring: #b45309;
  --hover: #fde2b8;
  --on: #1d3557;
}

.visually-hidden {
```


© 2026 559 Solutions LLC

```
  position: absolute;
  width: 1px;
  height: 1px;
  margin: -1px;
  padding: 0;
  overflow: hidden;
  clip-path: inset(50%);
  white-space: nowrap;
  border: 0;
}

.choices label:hover { background: var(--hover); }
.choices input:focus-visible + label { outline: 3px solid var(--ring); }
.choices input:checked + label { background: var(--on); color: white; }
```




*2 is checked (dark) and has keyboard focus (ring); the mouse is over 3 (`:hover`). The radios are invisible, yet Tab landed on the checked one, and Send produced `?pick=2`. When the inputs were set to `display: none` instead, the first Tab skipped the group and went straight to Send. (real: rendered in Chromium at 420 px)*

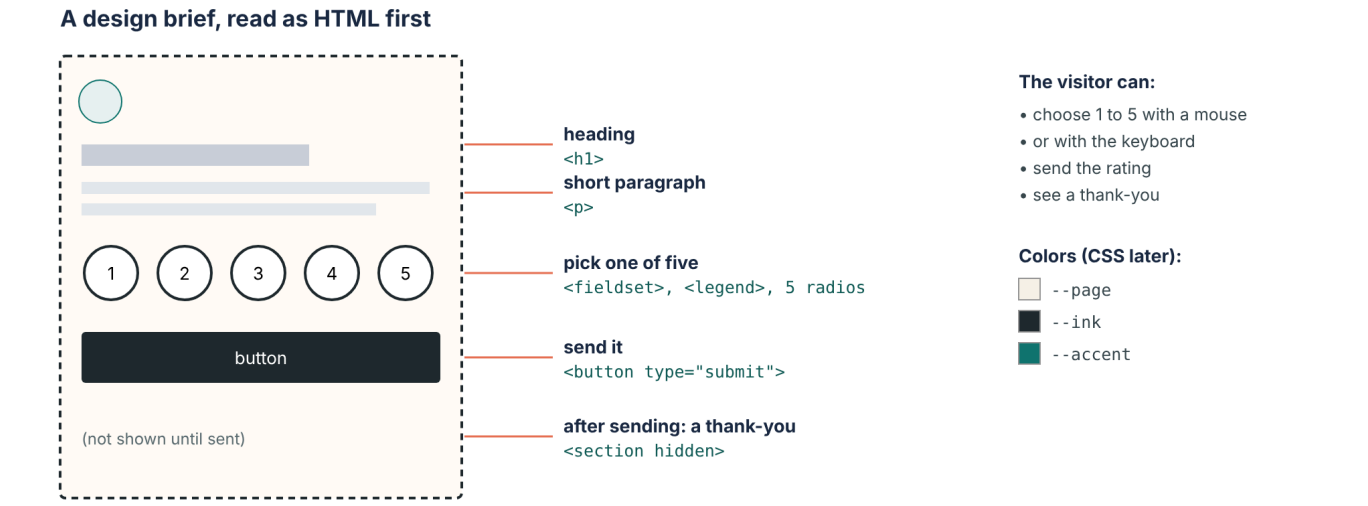
Predict: you swap the order of one input and its label in the HTML. Which state style stops working for that choice, and why does `+` explain it?

### 9. How do you read a design brief?

At work you rarely get code. You get a brief: a picture of the finished thing, what the visitor must be able to do, the words, and the colors.


© 2026 559 Solutions LLC

Read the brief twice. Then turn each part of the picture into an element, **before** you write any CSS. A heading is an `<h1>`. A choice of one from five is a `<fieldset>` with a `<legend>` and five radios that share a name. A button that sends is `<button type="submit">`. A message that appears later is a `<section hidden>`. Test that HTML with the keyboard and a real submit. If the page works with no CSS, it will keep working when the CSS is added. If it does not, CSS will only hide the problem.




*Reading a brief means naming the element for each piece of the picture. Colors wait for the CSS step.*

Look at the practice contact page you build in step 1 of the Exercise (`practice-04/contact.html`) and write its "brief" in three lines: what is on it, what the visitor can do, what it sends.

## Exercise

*Your hands, about 45 minutes (estimate).* Work in `practice-04`. Type it, do not paste it. `id`, `for`, `name` and `value` should go through your fingers once.

1. **Build a contact page.** Create `contact.html` with the normal head (charset, viewport, title). In the body, add `<h1>Contact me</h1>` and a `<form>`. Inside the form, write four labels, each followed by its field and joined to it with `for` and `id`, as in Concept 1:
    - **Your name:** an `<input type="text">` with `id="visitor-name"`, `name="visitor-name"` and `required`.
    - **Email:** the two lines from Concept 1.
    - **Nickname:** `<input type="text" id="nickname">` with **no** `name`. Leave it out on purpose.
    - **Message:** `<textarea id="message" name="message" required></textarea>`.


© 2026 559 Solutions LLC

End the form with `<button type="submit">Send message</button>`. Open `contact.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Your page will look plainer than the picture in Concept 1, which adds a little CSS; that is fine. Click each label's text: the cursor should jump into its field. Then fill in all four fields and press **Send message**. The address bar should end the way it does in the Concept 2 picture: three `name=value` pairs, and no nickname.

2. **Write the group.** Create `radio-practice.html` with the normal head (charset, viewport, title). In the body, type the form from Concept 3 inside a `<form>`, wrapped in `<fieldset>` with `<legend>How many hours did you code this week?</legend>`, followed by `<button type="submit">Send</button>`.

3. **Use the labels.** Open `radio-practice.html` in Edge or Chrome, the same way as in step 1. Click the words, not the circles. Each label selects its radio, and choosing one turns the others off.

4. **Send it.** Choose "3 to 5" and press Send. Look at the address bar: it should end in `?hours=3-5`. If it does not, check the `name` and `value` you typed.

5. **Keyboard only.** Press **F5**. Click an empty part of the page, then press Tab until the group has focus. Press the arrow keys. Press Tab again. Write one sentence: what do the arrow keys do inside the group, and what does Tab do?

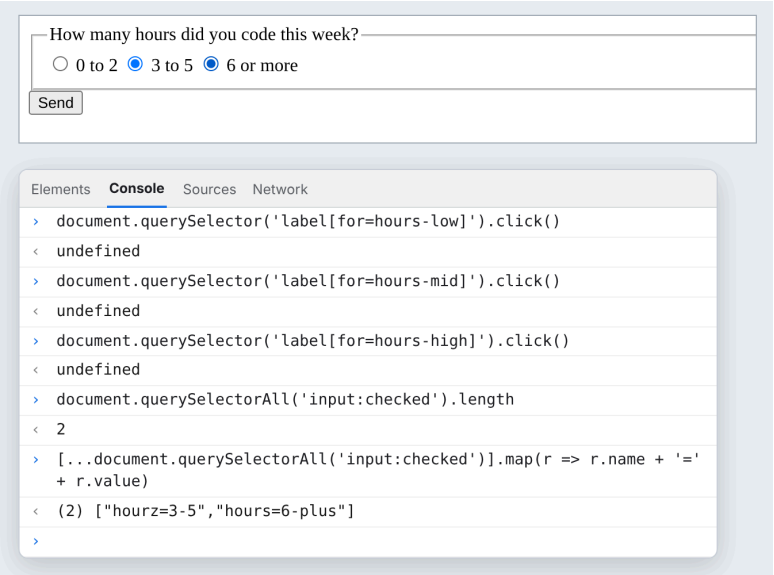
6. **Test `required`.** Press **F5**, so nothing is chosen, and press Send. The browser refuses to send, moves focus to the group and shows a short message, even though only the first radio says `required`. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Type this line exactly as shown. You will learn how it works in Module 9; for now, read the answer it gives.

   ```
   document.querySelector('form').checkValidity()
   ```

   It returns `false`. Choose any radio and type the line again.

7. **Break it on purpose.** Change the **second** radio's `name` from `hours` to `hourz`. Save, press **F5**, and click all three labels. Two radios stay on at once.


© 2026 559 Solutions LLC




*One typo made a second group. The Console confirms two checked radios, `hourz=3-5` and `hours=6-plus`. Submitting now would send both. You do not need to type these Console lines; they only show the check behind the picture, and a real Console prints the list with single quotes. (real: rendered in Chromium; Console values from a real run)*

Write one sentence: what changed, and why? Then put the name back to `hours`.

8. **Style it without breaking it.** Create `states.html` with the same kind of group (three choices, `name="pick"`). Give each input `class="visually-hidden"` and put each input directly before its label inside `<fieldset class="choices">`. Add a `<style>` element inside `<head>` (it holds CSS in the HTML file itself, instead of in a separate `styles.css`) with the CSS from Concept 8, plus a size and round shape for `.choices label` (for example `display: inline-block; width: 48px; height: 48px; border-radius: 50%; background: #eee; text-align: center; line-height: 48px;`). Open `states.html` in Edge or Chrome and test with the mouse, then with Tab and the arrow keys. The ring follows the keyboard.

9. **Break it again.** Add `.gone { display: none; }` to your `<style>`, then change one input's `class="visually-hidden"` to `class="gone"`. Save, press **F5**, and Tab into the group. Write one sentence: what happened to that choice for the keyboard? Put it back.

---

## Common misconception

> *"The radio circle is what the visitor clicks, so that is where the styling and the behavior go."*

What would you say? Hint: in step 3 of the Exercise, which element received your clicks? And in step 8, which element did the CSS turn into a round button?


© 2026 559 Solutions LLC

## Summary

A form holds controls, each joined to a `<label>` by `for` and `id`, and `required` stops the form until a control has a value. `id` is for the page and `name` is for the data. A form with no `method` sends `name=value` pairs in the address after a `?`, and a control with no `name` sends nothing. Radios with the same `name` form a group where only one can be on, and `required` on one radio covers the group. `<fieldset>` and `<legend>` give the group a title for people who cannot see it. `hidden` removes an element from sight, from screen readers and from the Tab order. Tab moves between controls and the arrow keys move inside a radio group. `:focus-visible` styles the keyboard focus ring, `aria-current="page"` marks the current nav link, and a skip link jumps past the menu. Custom properties on `:root` hold your colors, state selectors style the label from the radio's state, and `.visually-hidden` hides a control without breaking it. A brief is read twice and built as HTML first.

## Check your understanding

1. Only the first radio has `required`, but submitting with nothing selected is refused. What is `required` really attached to?
2. Your Contact form submits, but the message never appears in the address. Name the two most likely causes.
3. **What would you test** to prove that five labels are correctly joined to their five radios, without reading the HTML?

## Before next module

- [ ] `radio-practice.html` sends `?hours=…` and you wrote your two sentences from steps 5 and 7
- [ ] `states.html` works with the mouse and the keyboard, and you saw what `display: none` broke
- [ ] Every label on `practice-04/contact.html` focuses its field when clicked, and every field that should send data has a `name`
- [ ] You can say which nav link gets `aria-current="page"` on each page of a site, and how `nav a[aria-current="page"]` styles it
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

## Go further

- MDN: `<form>`; `<label>`; `<input type="radio">`; `<fieldset>`; the `hidden` global attribute; `:focus-visible`; *ARIA: aria-current attribute*; *Using CSS custom properties (variables)*; *Sending form data*


© 2026 559 Solutions LLC

- freeCodeCamp: the HTML forms and accessibility lessons in the current curriculum (search "accessibility")

---

## What to keep

Keep `practice-04` with all three files fixed, and your sentences from steps 5, 7 and 9. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

---

*Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.*

*Sources checked 2026-10-09: MDN "`<form>`: The Form element"; MDN "`<label>`: The Label element"; MDN "`<input type="radio">`"; MDN "`<fieldset>`: The Field Set element"; MDN "hidden" (global attribute); MDN ":focus-visible"; MDN "ARIA: aria-current attribute"; MDN "Using CSS custom properties (variables)"; MDN "Sending form data". Checked 2026-10-11: MDN "hidden" (global attribute, again); MDN ":root"; MDN "ARIA"; MDN "Attribute selectors"; MDN "position"; MDN "`<style>`: The Style Information element"; MDN "Color contrast"; MDN "outline-color"; MDN "clip" (deprecated; the page points to `clip-path`); MDN "clip-path". Submissions, validity checks and keyboard behavior tested in Chromium 141 (headless, via Playwright) on Linux. Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).*

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩


© 2026 559 Solutions LLC

PART 2

# Project: A rating card and a Contact page

*Your hands, about 2 hours (estimate).* In the Exercise you built a practice contact page and a styled radio group in `practice-04`. Now you add two pages to your real site. The first is the **rating card**: a small card that asks "Rate this lesson," offers five round choices from 1 to 5, and has a **Send rating** button. You build it from a written brief, HTML first, then CSS. The second is a **Contact** page with a real form, built the way you built your practice contact page. Both pages join the nav on every page, and the nav learns to mark the page you are on. You use only HTML and CSS. There is no JavaScript: the card's JavaScript comes in Module 10.

This project starts from the Module 3 packet's site: Home, About and Interests, one shared `styles.css` with a mobile-first header, and an `images` folder. If your site does not have that yet, finish the Module 3 packet project first (Part 2), then come back.

## The brief

Read the whole brief twice before you type anything. This is the kind of brief you get at work: a picture, what the visitor can do, the exact words, and the colors.

When you finish, `my-site` holds two new pages, `rate.html` and `contact.html`, next to `index.html`, `about.html`, `interests.html`, `styles.css`, `image-one.jpg` and the `images` folder. All five pages link the same `styles.css`.

**The nav.** Every page's nav has five links, in this order: **Home**, **About**, **Interests**, **Rate**, **Contact**. Rate goes to `rate.html`. Contact now goes to `contact.html` (in Modules 2 and 3 it went to `index.html#contact`). On each page, the link to that page has `aria-current="page"`, and no other link has it. So the headers are now the same on every page except for where that one attribute sits.

**The rating card page.** It has the same `<head>`, header and footer as your other pages, with the title `Rate this lesson - Your Name`. Its `<main>` holds one `<div class="rating-card">` with two parts:

- A `<form class="rating-form">` with an `<h2>` that says **Rate this lesson**, one sentence: **How clear was today's lesson? Choose 1 if you felt lost, 5 if it all made sense.**, then a `<fieldset class="choices">` and a `<button type="submit">` that says **Send rating**.
- The fieldset starts with a `<legend>` that says **Rate the lesson from 1 to 5**. Sighted visitors do not see it, but screen readers read it. Then come five radios, each right before its own label. All five share `name="rating"`. Their values are `1` to `5`, their ids `rating-1` to `rating-5`, and each label shows its number. The first radio has `required`.
- A `<section class="thanks" hidden>` after the form. It holds a paragraph **You chose out of 5**, an `<h2>` **Thanks for the feedback**, and one more sentence of your own. The check icon in the lesson's picture of the thank-you state is optional; the card works without it. The section stays hidden in this module. In Module 10, JavaScript fills the span, hides the form and shows this section.


© 2026 559 Solutions LLC

**What the visitor can do.** With a mouse: click a number to choose it, and see it change color when the pointer is over it. With a keyboard: press Tab to reach the group, use the arrow keys to choose, see a dark orange ring around the focused number, then Tab to **Send rating** and press Enter. If nothing is chosen, the browser refuses to send and says so. If 4 is chosen, the address ends in `?rating=4`.

**The colors.** Five custom properties on `:root`, used everywhere the card needs a color:

| Name | Value | Used for |
|---|---|---|
| `--card` | `#ffffff` | the card background |
| `--ink` | `#1d3557` | borders, number text, the button (the navy of your header) |
| `--accent` | `#0f766e` | a checked number, and the button on hover (teal) |
| `--soft` | `#e6f2f0` | a number under the mouse (pale teal) |
| `--ring` | `#b45309` | the focus ring (dark orange) |

The accent colors are your choice: pick your own `--accent` and `--soft`, as long as white text on `--accent` and `--ink` text on `--soft` both reach 4.5:1. `--ink` and `--ring` stay fixed, as in the table. The values above were checked with the WCAG contrast formula: white on the teal is 5.5:1, the navy on the pale teal is 10.8:1, the dark orange ring is 5.0:1 on the white card and 4.7:1 on your cream page background.

**The Contact page.** Same `<head>`, header and footer, with the title `Contact - Your Name`. Its `<main>` holds a `<section>` with an `<h2>` **Contact me**, one sentence with `class="lead"`, and a `<form class="contact-form">` with three labeled fields and a button:

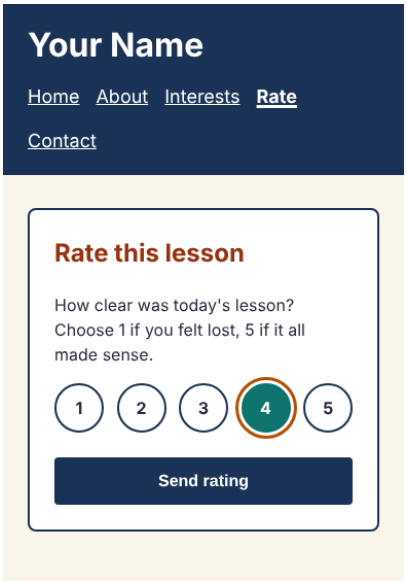
- **Your name:** `<input type="text">`, id and name `visitor-name`, `required`.
- **Email:** `<input type="email">`, id and name `visitor-email`, `required`.
- **Message:** `<textarea>`, id and name `message`, `rows="5"`, `required`.
- A `<button type="submit">` that says **Send message**.

Unlike the practice page, every field has a `name`. There is no nickname trap this time.

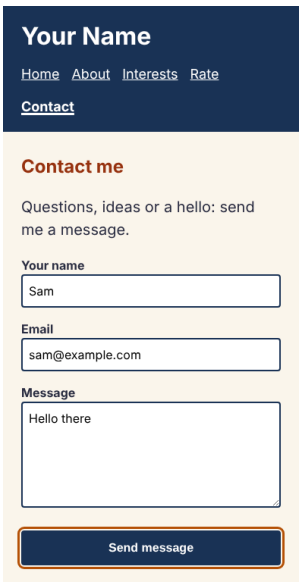
**Focus rings.** Every link, input, textarea and button gets a 3 px dark orange ring (`#b45309`) when it has keyboard focus. The nav links sit on the navy bar, where that ring is too dark (2.5:1), so they get the light orange ring `#fdba74` (7.3:1 on the navy). The link with `aria-current="page"` is bold, with a thicker underline.

On a phone, the card is as wide as the page allows, and the five numbers sit in one row. On a laptop, the card stays 400 px wide in the middle of the page. Nothing scrolls sideways at 390 px. The class names, ids, names, values and file names must match the brief; the sentences you write yourself are yours.


© 2026 559 Solutions LLC




*The finished rating card on a phone, driven by keyboard only: six Tabs, then three presses of the right arrow. 4 is checked (teal) and has keyboard focus (dark orange ring). Rate is bold in the nav because this page's link has `aria-current="page"`. (real: rendered in Chromium at 390 px)*




*The Contact page filled in, with the Send message button reached by Tab. Each label sits above its field. (real: rendered in Chromium at 390 px)*

## The steps

Type every line. Do not paste. The one exception is the header and footer, which you copy from an existing page, as you did in Modules 1 and 3.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.


© 2026 559 Solutions LLC

2. **Read the brief like a builder.** On paper, sketch the card. Next to each part, write the element it becomes, as in the lesson's Concept 9: the heading, the sentence, the group of five, the button, the thank-you. Do not write any CSS yet. Then write down one prediction: when you submit the card with 4 chosen, what will the end of the address say?

3. **Update the nav on the three old pages.** In `index.html`, change the nav to five links and mark the current page:

   ```
   <nav>
     <a href="index.html" aria-current="page">Home</a>
     <a href="about.html">About</a>
     <a href="interests.html">Interests</a>
     <a href="rate.html">Rate</a>
     <a href="contact.html">Contact</a>
   </nav>
   ```

   Make the same change in `about.html` and `interests.html`, but move `aria-current="page"` to that page's own link: About on the About page, Interests on the Interests page. Then read the three navs line by line. They must match except for where `aria-current` sits. If you already added `aria-current` after the lesson, check it and add the two new links. Your Home page can keep its old Contact section; the nav simply does not point to it any more.

4. **Build the rating card HTML.** In the Explorer, create `rate.html`. Copy all of `about.html` into it. Change the title to `Rate this lesson - Your Name`, move `aria-current="page"` to the Rate link, and replace everything between `<main>` and `</main>` with this skeleton:

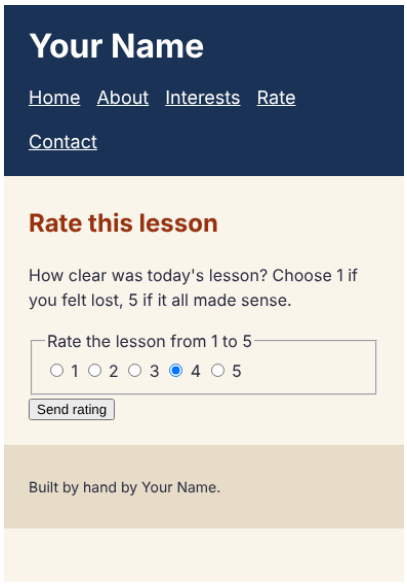
   ```
   <main>
     <div class="rating-card">
       <form class="rating-form">
         <h2>Rate this lesson</h2>
         <p>How clear was today's lesson? Choose 1 if you felt lost, 5 if it all
   made sense.</p>
         <fieldset class="choices">
           <legend class="visually-hidden">Rate the lesson from 1 to 5</legend>
           <input class="visually-hidden" type="radio" name="rating" value="1"
   id="rating-1" required>
           <label for="rating-1">1</label>
           <input class="visually-hidden" type="radio" name="rating" value="2"
   id="rating-2">
           <label for="rating-2">2</label>
           <!-- Continue alone: radios 3, 4 and 5, each input right
                before its label, the same pattern as 2. -->
         </fieldset>
         <button type="submit">Send rating</button>
       </form>
   ```


© 2026 559 Solutions LLC

```
    <!-- Continue alone: the section with class="thanks" and the
         hidden attribute, with the three parts from the brief. -->
  </div>
</main>
```

Read one radio to yourself, out loud: what do its `id`, `name` and `value` each do, and which label points to it? The class `visually-hidden` does nothing yet. Its rule comes in step 7, so for now you see plain radios and the legend.

5. **Test the HTML before any CSS.** Save, open `rate.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window), and press **F5** after each save. The card has no look yet: plain circles in a box with the legend in its border.




*The card's HTML with only the old site styles. It already works: the legend is visible, the radios are real, and 4 is selected. (real: rendered in Chromium at 390 px)*

Now test it four ways, and tick each one:

- [ ] **Labels.** Click the number 3, not the circle. Circle 3 turns on. Click 5: 3 turns off.
- [ ] **Keyboard.** Press F5, put the mouse aside, and press Tab. Chromium visited the five nav links, then the group (circle 1, with nothing chosen), then **Send rating**. Inside the group, the arrow keys move the choice.
- [ ] **Nothing chosen.** Press F5 and press **Send rating**. The address does not change, focus moves to circle 1, and a small message appears (Chromium's says "Please select one of these options."; yours may be worded differently).
- [ ] **A real submit.** Choose 4 and press Send rating. The address now ends in `rate.html?rating=4`. Compare it with your prediction from step 2.

If one test fails, fix the HTML now. CSS will not fix it later; it will only hide it.


© 2026 559 Solutions LLC

6. **Add the focus rings and the current-page style.** Open `styles.css`. In the site-wide basics, after the `a` rule, add one rule for every control a keyboard can reach:

```
a:focus-visible,
input:focus-visible,
textarea:focus-visible,
button:focus-visible {
  outline: 3px solid var(--ring);
  outline-offset: 2px;
}
```

A comma between selectors means "the same rule for each of these." `var(--ring)` reads a custom property you define in the next step. Then, in the header and nav group, after your `nav a` rule:

```
nav a:focus-visible {
  outline-color: #fdba74;
}

nav a[aria-current="page"] {
  font-weight: 700;
  text-decoration-thickness: 3px;
}
```

`outline-color` swaps only the ring's color on the navy bar; it keeps the width and style. `text-decoration-thickness` makes the underline thicker.

7. **Add the colors and the hiding class.** At the top of `styles.css`, right after the box-sizing rule and before the site-wide basics, add the colors from the brief:

```
/* Colors: custom properties */
:root {
  --card: #ffffff;
  --ink: #1d3557;
  --accent: #0f766e;
  --soft: #e6f2f0;
  --ring: #b45309;
}
```

Then, at the end of the reusable classes group, add the class from Concept 8, exactly:

```
/* Hidden from sight, still there for keyboards and screen readers */
.visually-hidden {
  position: absolute;
```


© 2026 559 Solutions LLC

```
  width: 1px;
  height: 1px;
  margin: -1px;
  padding: 0;
  overflow: hidden;
  clip-path: inset(50%);
  white-space: nowrap;
  border: 0;
}
```

Save and press F5 on `rate.html`. The circles and the legend disappear, and only the numbers are left. Press Tab until you pass the nav: focus still enters the group, even though you cannot see the radios. Chromium measured each hidden radio at 1 × 1 px.

8. **Style the card.** Add a new group after the Interests page group and **before** the wider-screens block, which stays last:

```
/* Rating card */
.rating-card {
  max-width: 400px;
  margin: 24px auto;
  padding: 24px;
  background-color: var(--card);
  border: 2px solid var(--ink);
  border-radius: 8px;
}

.rating-card h2 {
  margin-top: 0;
}

.choices {
  border: 0;
  margin: 0 0 24px;
  padding: 0;
  display: flex;
  justify-content: space-between;
}

.choices label {
  width: 48px;
  height: 48px;
  display: grid;
  place-items: center;
  border: 2px solid var(--ink);
  border-radius: 50%;
```


© 2026 559 Solutions LLC

```
  font-weight: 700;
  cursor: pointer;
}

.choices label:hover {
  background-color: var(--soft);
}

.choices input:focus-visible + label {
  outline: 3px solid var(--ring);
  outline-offset: 3px;
}

/* Continue alone: the checked state. A label right after a checked
   input gets var(--accent) as its background and border color,
   and white text. */
```

`margin: 24px auto` puts the card in the middle once the page is wider than 400 px. `display: grid` with `place-items: center` puts the number in the middle of its circle. The `border: 0` and `padding: 0` on `.choices` remove the box every fieldset has by default. The legend is still there for screen readers; only the box goes.

Then style the button. Both forms use the same button, so one rule covers both:

```
.rating-form button,
.contact-form button {
  width: 100%;
  padding: 12px;
  font-size: 16px;
  font-weight: 700;
  color: white;
  background-color: var(--ink);
  border: 2px solid var(--ink);
  border-radius: 4px;
  cursor: pointer;
}

/* Continue alone: the same two selectors with :hover, using
   var(--accent) for the background and the border. Then a .thanks
   rule with text-align: center. Give .thanks no display value:
   a display rule would beat the hidden attribute. */
```

9. **Run the card.** Save, press F5, and check it against the picture in the brief. Then repeat the four tests from step 5 with the new look: labels, keyboard, nothing chosen, a real submit. This time, watch the ring: it follows the keyboard onto the numbers and the button, and it is light orange on


© 2026 559 Solutions LLC

the nav links. Move the mouse over a number to see the pale hover color.

10. **Peek at the thank-you, then hide it again.** In `rate.html`, delete the word `hidden` from the thanks section and save. Press F5: the thank-you section appears under the form, with an empty space where the number will go. Put `hidden` back, save, and press F5. In Module 10, JavaScript does this swap for you.

11. **Build the Contact page.** Create `contact.html`. Copy all of `about.html` into it, change the title to `Contact - Your Name`, move `aria-current="page"` to the Contact link, and replace the inside of `<main>`:

```
<main>
  <section>
    <h2>Contact me</h2>
    <p class="lead">One sentence that invites a message.</p>
    <form class="contact-form">
      <label for="visitor-name">Your name</label>
      <input type="text" id="visitor-name" name="visitor-name" required>

      <!-- Continue alone: the Email label and input, the Message
           label and textarea (rows="5"), and the Send message
           button, as in the brief. Every field gets a name. -->
    </form>
  </section>
</main>
```

12. **Style the Contact form.** After the rating card group, and still before the wider-screens block, add:

```
/* Contact page */
.contact-form {
  max-width: 480px;
}

/* Continue alone: .contact-form label is a block with bold text
   and 16px space above it. .contact-form input and .contact-form
   textarea are 100% wide, with 8px padding, font: inherit, a 2px
   border in var(--ink) and a 4px radius. .contact-form button gets
   24px space above it. */
```

`font: inherit` matters: browsers give inputs and textareas their own smaller font unless you tell them to use the page's.


© 2026 559 Solutions LLC

13. **Test the Contact page.** Save and press F5. Click each label: the cursor jumps into its field. Press F5 and press **Send message** with everything empty: the browser stops at Your name. Fill in all three fields and send. On the sample page, Chromium's address ended in:

    ```
    contact.html?visitor-name=Sam&visitor-
    email=sam%40example.com&message=Hello+there
    ```

    Three pairs, because all three fields have a `name`.

14. **Modify.** In `:root`, change `--accent` to another dark color, for example `#7c3aed`. Save and press F5 on `rate.html`. The checked number and the button's hover color both change, from one line. Check that white text is still easy to read on your new color, then put `#0f766e` back (or keep your color if it reaches 4.5:1 with white).

15. **Break it on purpose: `display: none`.** Add this rule to the reusable classes group, then change all five `class="visually-hidden"` on the **radios** (not the legend) to `class="gone"`:

    ```
    .gone {
      display: none;
    }
    ```

    Save, press F5, and predict before you press Tab. The card looks the same. Now Tab through the page. Chromium went from the Contact link straight to **Send rating**: the group was skipped. The mouse still works, because clicking a label still checks its radio. Press F5 and press Send rating with nothing chosen. Nothing happens at all: no message, no change in the address. Open the Console (**F12**, then the **Console** tab). Chromium printed `An invalid form control with name='rating' is not focusable.` (once for each radio, so you may see it five times) The browser wants to send focus to the group to show its message, and it cannot. Put `class="visually-hidden"` back on all five radios and delete the `.gone` rule.

16. **Break it on purpose: one wrong name.** Change the second radio's `name="rating"` to `name="ratng"`. Save, press F5, click 2, then click 4. Both stay teal. Press Send rating. Chromium's address ended in `?ratng=2&rating=4`: two groups, two pairs. Put the name back.

17. **Debug: a number that will not turn on.** Change the fourth label to `<label for="rating4">` (drop the dash). Save and press F5. Click 4. Nothing turns teal. Click 5, then 4 again: 5 stays on. Before you look at the code, write down which part is broken: the radio, the label, or the CSS. Then find it. The label's `for` points to an id that does not exist, so the click goes nowhere, and nothing is shown in the Console. The test from Check your understanding 3 in the lesson is how you catch this: click every label, one by one. Fix the `for` and test all five.

18. **Test the site.** Tick each box, at 390 px in device mode (F12, then Ctrl+Shift+M) and in a full-width window.


© 2026 559 Solutions LLC

- [ ] All five pages show the same five nav links, and each page's own link is bold
- [ ] Every nav click works: five links on five pages
- [ ] Nothing scrolls sideways at 390 px on any page
- [ ] With the keyboard only, you can choose a rating and send it; the ring is dark orange on the card and light orange on the nav
- [ ] Send rating with nothing chosen is refused; with 4 chosen, the address ends in `?rating=4`
- [ ] The thank-you section is not visible
- [ ] Every Contact label puts the cursor in its field, and a full submit shows three pairs in the address

19. **Put it online.** Go to Part 3, *Finishing work in Git*, and publish both pages with the save routine. Then run the test from step 18 again on the live site.


© 2026 559 Solutions LLC

PART 3

# Finishing work in Git

Your rating card and Contact page work on your computer. Now you put them online with the same save routine: `git status`, `git add .`, `git commit`, `git push`, then check the live page. What is new this time is two new pages in one commit, and testing a live form with the keyboard.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 3.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **See what changed.** Type:

   ```
   git status
   ```

   Under **Changes not staged for commit** you should see `modified:` for `about.html`, `index.html`, `interests.html` and `styles.css`: the three old pages changed because of the nav. Under **Untracked files** you should see `contact.html` and `rate.html`. If one of the three old pages is missing from the list, you forgot to update its nav. Fix it before you go on.

3. **Stage the changes.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. Run `git status` again. Under **Changes to be committed** you should now see six lines: four `modified:` and two `new file:` (`contact.html` and `rate.html`).

4. **Commit (NEW: two pages in one commit).** Type:

   ```
   git commit -m "Add a rating card page and a Contact page"
   ```

   On the sample site, Git printed `6 files changed` on the second line, then one `create mode 100644` line for each new page. Your commit id and line counts will differ. The two pages share the new nav, the focus rings and the colors in `styles.css`, so they go into one commit. A nav with a link to a page that is not there yet would be a broken site.

5. **Push.** Type:

   ```
   git push
   ```

   The last line ends in `main -> main`.


© 2026 559 Solutions LLC

6. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 3 commit under it.

7. **Check the live pages.** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/rate.html` and press F5. Click each of the five nav links and check that each page's own link is bold.

8. **Test the live forms with the keyboard (NEW).** On the live `rate.html`, put the mouse aside. Tab to the group, choose 4 with the arrow keys, Tab to **Send rating** and press Enter. The address should end in `rate.html?rating=4`. The page looks the same after the submit: GitHub Pages only serves files, so nothing receives or stores the rating. In Module 10, the card's JavaScript shows the thank-you section on the page instead. Do the same on the live `contact.html` with all three fields filled in.

9. **Test on a phone.** If you have a phone, open your live Rate page on it. Tap a number, tap **Send rating**, and read the address.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. On the live site, all five nav links work on all five pages, a keyboard-only rating sends `?rating=4`, and the Contact form sends three pairs.


© 2026 559 Solutions LLC

PART 4

# Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose on the sample site and tested in Chromium. After each fix, save, press F5, and run the test from Project step 18 again.

### 1. Clicking a label does nothing

**Symptom.** On the rating card, clicking one number does not turn it teal, and the choice you made before stays on. Or, on the Contact page, clicking a label does not put the cursor in its field. There is no error in the Console.

**Why.** The label's `for` does not match the control's `id`, letter for letter. On the sample card, `<label for="rating4">` next to `id="rating-4"` left the label joined to nothing: clicking 4 checked no radio, and after clicking 5, clicking 4 left 5 on. A screen reader user has the same problem in a worse form: the control has no name read out at all.

**Fix.** Click every label, one by one. For the one that fails, compare its `for` with the `id` of the control right before it (rating card) or right after it (Contact page). Look for a missing dash, a capital letter, or a number that was not changed when you copied a line.

### 2. Two numbers stay on at the same time

**Symptom.** You click 2, then 4, and both stay teal. Sending puts two pairs in the address.

**Why.** The radios are not one group. The `name` makes the group, so one radio with a different name starts a group of its own. On the sample card, `name="ratng"` on the second radio gave `rate.html?ratng=2&rating=4`.

**Fix.** Check that all five radios say exactly `name="rating"`. Then click 1 to 5 in turn: each click must turn the one before it off.

### 3. A field is missing from the address

**Symptom.** You fill in the whole Contact form and press Send message. The address shows `visitor-name` and `visitor-email`, but not `message`.

**Why.** A control with no `name` is not sent. On the sample page, a textarea without `name="message"` gave `contact.html?visitor-name=Sam&visitor-email=sam%40example.com`. A radio with a `name` but no `value` sends the word `on` (MDN, `<input type="radio">`), so you cannot tell which number was chosen.

**Fix.** Every control that should send data needs a `name`; every radio also needs its own `value`. Remember the split from the lesson: `id` is for the page (the label, the CSS), `name` is for the data.


© 2026 559 Solutions LLC

### 4. The focus ring is hard to see on the nav

**Symptom.** You Tab through the nav and the ring around each link is a dark, muddy line on the navy bar. You can hardly tell which link has focus.




*Without the nav rule, the Interests link gets the dark orange ring meant for light backgrounds: 2.5:1 against the navy. (real: rendered in Chromium at 390 px)*




*With `nav a:focus-visible { outline-color: #fdba74; }` the ring is light orange: 7.3:1 against the navy. (real: rendered in Chromium at 390 px)*

**Why.** The site-wide rule gives every focused link a `#b45309` ring. That is 5.0:1 on white, but only 2.5:1 on `#1d3557`, below the 3:1 that MDN's *Color contrast* asks for interface parts. The `nav a:focus-visible` rule is missing or misspelled. Its place in the file does not matter: `nav a:focus-visible` names one more element than `a:focus-visible`, so it has higher specificity (Module 2) and wins wherever it sits.

**Fix.** In the header and nav group, check for exactly:

```
nav a:focus-visible {
  outline-color: #fdba74;
}
```

Then Tab through the nav again. Each ring should be light orange.


© 2026 559 Solutions LLC

### 5. The keyboard skips the rating group

**Symptom.** The card looks right and works with the mouse. But Tab goes from the Contact link straight to **Send rating**. Pressing Send rating with nothing chosen does nothing at all: no message, no change in the address.

**Why.** The radios were hidden with `display: none` instead of the `.visually-hidden` class. That takes them out of the Tab order, and the browser can no longer move focus to the group to show its "please select" message. On the sample card, Chromium printed this in the Console each time Send rating was pressed (once for each radio, so you may see it five times):

```
An invalid form control with name='rating' is not focusable.
```

The same happens if the `.visually-hidden` rule itself has `display: none` in it.

**Fix.** Give each radio `class="visually-hidden"`, and check that the rule is the one from Project step 7, with `clip-path: inset(50%)` and no `display` line. Then press Tab: focus should land on the group, and the arrow keys should move the teal number.


© 2026 559 Solutions LLC

PART 5

# Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 5. If a line is a 1 or a 2, reread that Core Concept and redo its Exercise or Project step.

| Skill | 1 Beginning | 2 Developing | 3 Proficient | 4 Ready to advance |
|---|---|---|---|---|
| I can join every label to its control with for and id, and test it by clicking the label | ☐ | ☐ | ☐ | ☐ |
| I can explain the difference between id and name, and predict the name=value pairs a form will put in the address | ☐ | ☐ | ☐ | ☐ |
| I can build a radio group with one shared name, a value on each radio, and required on one radio that covers the group | ☐ | ☐ | ☐ | ☐ |
| I can wrap a group in fieldset and legend, and keep the legend for screen readers with .visually-hidden | ☐ | ☐ | ☐ | ☐ |
| I can use the hidden attribute and say what it removes: sight, screen readers and the Tab order | ☐ | ☐ | ☐ | ☐ |
| I can move through a form with Tab, Shift+Tab, the arrow keys and Enter, and say where focus should go | ☐ | ☐ | ☐ | ☐ |
| I can add :focus-visible rings that reach 3:1 against what is behind them, and mark the current nav link with aria-current="page" | ☐ | ☐ | ☐ | ☐ |
| I can turn a written brief into HTML elements before any CSS | ☐ | ☐ | ☐ | ☐ |
| I can store colors in custom properties on :root and style a hidden radio's label with :hover, :checked and :focus-visible | ☐ | ☐ | ☐ | ☐ |

## Three questions to answer in writing

1. In Project step 15, `display: none` on the radios made the Send rating button do nothing when no number was chosen. Explain why, in your own words, and say what `.visually-hidden` keeps that `display: none` takes away.
2. Pick one mistake you made in this module. What did the page, the address bar or the Console show, and how did you find the cause?


© 2026 559 Solutions LLC

3. Your Contact page needs a fourth field: a "How did you find me?" choice of Search, A friend or GitHub. List every element and attribute you would write, in order, and say what the address would end in if a visitor chose A friend.


© 2026 559 Solutions LLC

PART 6

# Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

## Check your understanding

1. **What `required` is attached to.** It is written on one radio, but for radios it works on the whole **group**: every radio with the same `name`. MDN: if any radio in a same-named group has `required`, "a radio button in that group must be checked, although it doesn't have to be the one with the attribute applied." In the lesson's Console check, the third radio had no `required` but still reported `valueMissing: true`. On your rating card, only `rating-1` has `required`, and Send rating with nothing chosen was refused. (Core concept 3; Exercise step 6; Project step 5)
2. **The message never appears in the address.** The two most likely causes: the textarea has **no `name`** (a control with no name is not sent; on the sample page the address then ended after `visitor-email`), or the textarea sits **outside the `<form>` tags**, so it is not part of the form. (An aside: an empty field that is not required still sends its name with nothing after it, `message=`. Your Contact form's textarea is `required`, so the browser will not send it empty.) (Core concept 2; *Troubleshooting*, problem 3)
3. **Proving five labels are joined.** Click each label's text, one by one, with the mouse, not the circles. Each click must check its own radio and turn the one before off. Then send the form after choosing each number and read the address: `?rating=1` to `?rating=5`. A label with a wrong `for` checks nothing (Project step 17). With the keyboard you can test the group, but not the labels, because the arrow keys move between radios without using the labels. (Core concepts 1 and 3; Project steps 5 and 17)

## Common misconception

*"The radio circle is what the visitor clicks, so that is where the styling and the behavior go."* In step 3 of the Exercise you clicked the **label**, the words, and the label checked its radio through `for` and `id`. In step 8, the CSS turned the label into the round button, and the radio itself became a 1-pixel, invisible control. The behavior (the group, the arrow keys, `required`, the `name=value` that is sent) still belongs to the real radio. The look belongs to the label, styled from the radio's state with `input:checked + label` and `input:focus-visible + label`. Your rating card works the same way: you never see a radio, yet Tab, the arrow keys and `?rating=4` all come from the radios. (Core concept 8; Project steps 7 and 8)


© 2026 559 Solutions LLC

## Exercise checks

- **Step 1.** Clicking each label puts the cursor in its field. After a full submit, the address ends in `?visitor-name=…&visitor-email=…&message=…`: three pairs, and no nickname, because that input has no `name`. `@` shows as `%40` and a space as `+`. (Core concepts 1 and 2)
- **Steps 3 and 4.** Clicking a label selects its radio and turns the others off. With "3 to 5" chosen, the address ends in `?hours=3-5`. (Core concept 3)
- **Step 5.** Sample sentence: "The arrow keys move the choice inside the group; Tab leaves the group and goes to the next control." The first Tab into the group focuses it without choosing anything. (Core concept 6)
- **Step 6.** Send with nothing chosen is refused, focus moves to the group, and `checkValidity()` returns `false`. After you choose any radio, it returns `true`. (Core concept 3)
- **Step 7.** With `hourz` on the second radio, two radios can be on at once, because a different `name` makes a second group. (Core concept 3)
- **Step 8.** The mouse colors a choice on hover and fills it when checked. With the keyboard, the ring sits on the label of the focused radio and moves with the arrow keys. (Core concept 8)
- **Step 9.** Sample sentence: "With `display: none`, that radio left the Tab order, so the keyboard could no longer reach that choice." (Core concepts 5 and 8; *Troubleshooting*, problem 5)

## Self-assessment questions

1. **Why `display: none` broke the Send rating button.** `display: none` takes the radios out of the page, so the keyboard cannot reach them and nothing can focus them. The first radio has `required`. When you press Send rating with nothing chosen, the browser refuses to send, and it wants to move focus to the group to show its message. It cannot focus a radio with `display: none`, so it shows nothing at all. The Console says `An invalid form control with name='rating' is not focusable.` `.visually-hidden` keeps each radio on the page, only 1 × 1 px. So it stays in the Tab order, it can take focus, a screen reader can still read it, and the browser can show its message. (Project steps 5, 7 and 15)
2. **A mistake you made.** There is no single answer. A good answer names the mistake, quotes what the page, the address bar or the Console really showed, for example `?ratng=2&rating=4`, and says how you found the cause, for example by clicking every label one by one.
3. **A "How did you find me?" field.** First, a `<fieldset>`. Second, a `<legend>` that says **How did you find me?** Third, three radios, each right before its own label: `<input type="radio" name="found-by" value="search" id="found-search">` with `<label for="found-search">Search</label>`, then the same for `friend` with the label **A friend**, and for `github` with the label **GitHub**. All three share one `name`, each has its own `id` and `value`, and each label's `for` matches its radio's `id`. If you put the fieldset after the Message field and a visitor chose A friend, the address would end in `&found-by=friend`: the group's `name`, then the checked radio's `value`. Your `name` and `value` words may differ; the pattern is what matters. (The brief; Project steps 4, 5, 13, 16 and 17)


© 2026 559 Solutions LLC

## Project checks

- **Step 2.** The address ends in `rate.html?rating=4`: the group's `name`, then the checked radio's `value`.
- **Step 5.** Tab order in Chromium: the five nav links, the group (radio 1, nothing chosen), then Send rating. The empty submit leaves the address unchanged and moves focus to radio 1.
- **Step 7.** The radios and the legend disappear, but Tab still reaches the group. Each hidden radio measures 1 × 1 px.
- **Step 8, continue alone.** The checked state:

  ```
  .choices input:checked + label {
    background-color: var(--accent);
    border-color: var(--accent);
    color: white;
  }
  ```

  The button hover and the thanks rule:

  ```
  .rating-form button:hover,
  .contact-form button:hover {
    background-color: var(--accent);
    border-color: var(--accent);
  }

  .thanks {
    text-align: center;
  }
  ```

- **Step 10.** Without `hidden`, the thank-you shows under the form with an empty space where the number goes. With `hidden`, Chromium measured the section at height 0 with `display: none`. Adding `display: block` to `.thanks` in the CSS made it show (144 px tall) even with `hidden` in the HTML: that is why the brief says to give `.thanks` no `display`.
- **Step 12, continue alone.**

  ```
  .contact-form label {
    display: block;
    font-weight: 700;
    margin-top: 16px;
  }

  .contact-form input,
  ```


© 2026 559 Solutions LLC

```
.contact-form textarea {
  width: 100%;
  padding: 8px;
  font: inherit;
  border: 2px solid var(--ink);
  border-radius: 4px;
}

.contact-form button {
  margin-top: 24px;
}
```

- **Step 13.** The empty submit stops at Your name. A full submit gives `contact.html?visitor-name=Sam&visitor-email=sam%40example.com&message=Hello+there` with the sample text.
- **Step 14.** One change in `:root` changes every place that uses `var(--accent)`. White on `#7c3aed` is 5.7:1, so that color passes too.
- **Step 15.** Tab skips the group; the mouse still works; an empty Send does nothing, and the Console says `An invalid form control with name='rating' is not focusable.`
- **Step 16.** `?ratng=2&rating=4`.
- **Step 17.** The broken part is the label: its `for` points to `rating4`, an id that does not exist.

---

NEXT

**Module 5: JavaScript Primer.**
Free lesson: [559solutions.com/code559/module-05-javascript-primer/](559solutions.com/code559/module-05-javascript-primer/)


© 2026 559 Solutions LLC

# About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

## The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

## Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

**No jargon. No drama. Just solutions that serve the mission.**

— J. R. Wells

559solutions.com


© 2026 559 Solutions LLC